

Toward Quasi-Polynomial Unknot Recognition

An exact three-braid backend with linear symbolic cost,
certified Markov descent, and intrinsic scanner multiplicities

Research continuation for Vladimir Reshetnikov’s ProveIt project

7 October 2026

Abstract

We extend ProveIt’s exact unknot-recognition implementation, `fastunknot`. A supplied three-strand braid can be decided by reducing a word in $C_2 * C_3$, retaining its exponent sum to resolve the central ambiguity. The resulting complete specialized algorithm uses $O(\ell)$ symbol operations on an explicit word of length ℓ ; a conservative bit bound is $O(\ell \log(\ell + 2))$. An independent integer-matrix implementation checks the same classification in $O(\ell^2)$ elementary bit operations. We integrate these procedures with a general braid-writhe obstruction, replayable elementary Markov reductions, and the existing exact fallback.

We also identify the object multiplicities that remain after exhaustive categorical cancellation: they are Betti numbers of a scalar residue complex, and are independent of pivot choices at a fixed prefix. A diagnostic verifies this identity against the actual scanning engines. For disk frontiers, a nilpotent radical gives a finite homological perturbation formula for batch cancellation. We derive explicit width–multiplicity complexity bounds and explain why they are conditional on a separate multiplicity bound.

The topological classification and algebraic tools used here are established mathematics. The contributions are their convention-checked specialization, implementation, certification, cost accounting, and experimental evaluation. We do not obtain a general $n^{O(\log n)}$ recognizer. The report locates the remaining obligations in the Khovanov and geometric-hierarchy routes, taking account of both relevant 2026 preprints.

Contents

1	Outcome, scope, and the exact baseline	3
1.1	What is and is not a novelty claim	3
1.2	Input and computational model	4
2	The relevant state of the art	4
3	A complete three-braid classifier by free-product reduction	4
3.1	The topological input	5
3.2	An explicit constant-alphabet algorithm	5
4	An independent integer-matrix criterion	6
4.1	The determinant interpretation and a useful trap	7
4.2	Bit complexity and serialization	7

5	Extending the useful input class by certified preprocessing	8
5.1	A linear braid-writhe obstruction	8
5.2	Endpoint singleton destabilizations	8
5.3	A necessary negative control: Morton’s example	9
5.4	A closure certificate is not a local tangle replacement	9
6	Integration and exact semantics	9
6.1	Code delivered	10
7	Intrinsic multiplicities of a partial scanning complex	10
7.1	The scalar residue functor	11
7.2	What pivot optimization can change	11
7.3	Implemented diagnostic and its limitation	12
8	Finite batch cancellation through the radical	12
8.1	A conservative explicit arithmetic bound	13
9	What a width–multiplicity bound really proves	14
9.1	An explicit fixed-width obstruction	14
9.2	Decision, output size, and compressed multiplicity	15
10	Experiments and verification	15
10.1	Independent correctness checks	15
10.2	Paired performance measurements	16
10.3	Deterministic complexity evidence	17
10.4	Limits of the experimental evidence	18
11	The geometric route to the quasi-polynomial target	18
12	Research questions and proposed next work	19
12.1	Recover certified small-braid descriptions from PD inputs	19
12.2	Go beyond singleton endpoint reductions	19
12.3	A complete structural decision route for four-braids	19
12.4	Use intrinsic multiplicity to choose scan orders	20
12.5	Implement and compare radical batching	20
12.6	Compress differential operators, not only ranks	20
12.7	Resolve the multi-boundary matching model explicitly	20
12.8	Prove a useful multiplicity-profile theorem	20
12.9	Implement one complete geometric hierarchy operation	20
12.10	Strengthen certificates and formalize the small trusted core	21
13	Reproduction and integration	21
14	Conclusions	21

1 Outcome, scope, and the exact baseline

The concrete improvement is a complete inexpensive decision route for a verified low-strand braid representation, before the general pipeline has to build a Khovanov complex. On other braid inputs we add two certified opportunities to finish early: a writhe obstruction and explicit endpoint destabilizations. For planar-diagram and grid inputs, the existing exact recognizer remains available. We do not infer a low braid index from a small scan frontier.

The baseline is the **fast/** subtree at ProveIt commit

4e6fe879e5238ec0b134b6d33fdca0b8c7c1c711.

Its Git tree is b99a666269f660d932fc82ac8d1f29d3ae42231f. The last commit affecting the larger unknot-recognition subtree, when inspected, was 78333302d9babf0efb1a5d218e122f13c35bb306. The supplied provenance file records the source and hashes; this is a continuation of the integrated implementation, not a comparison against the earlier 0.1 prototype.

That distinction matters. The current baseline already contains Reidemeister I/II reduction, a bounded Reidemeister III search, descending tests, modular Alexander and bracket filters, visible connected-sum factorization, improved scan ordering, bit-packed cobordisms, interned matchings, cached gluing plans, and several pivot and racing strategies. Repeating those optimizations would not address the next structural bottleneck. The preceding nine proposals and synthesis are acknowledged as prior project work [1].

Result	Precise scope	Status in this bundle
Three-braid decision	Explicit validated braid word on at most three strands	Proved and implemented
General writhe rejection	Unknotted m -braids satisfy $ e \leq m - 1$	Proved from a cited theorem; implemented
Endpoint Markov descent	A sufficient, replayable destabilization criterion	Proved and implemented
Canonical multiplicities	A fixed prefix and a fixed homological normalization	Proved and checked on actual complexes
Finite radical batching	Disk arc algebra over $\mathbb{F}_2$	Proved; batching implementation pending
General quasi-polynomial decision	Arbitrary classical knot diagrams	Not achieved by this work

1.1 What is and is not a novelty claim

Polynomial algorithms for three-braid conjugacy, the classification of closed three-braids, and polynomial special-purpose computations are classical or already in the literature [5, 7, 8, 4]. The residue and perturbation arguments specialize established homological algebra [12, 15]. We present full proofs of the specific criteria and bounds needed by this implementation, rather than claim discovery of those general theories.

The new project-level outcome is an executable, independently checked route that avoids explicit homological bases on a complete input class, together with a diagnostic that tells us which multiplicities pivot tuning cannot remove. These are concrete performance improvements and a better specified research program. Their value does not depend on attaching an unsupported claim of a general breakthrough.

1.2 Input and computational model

A braid word is an explicit list $\beta = \sigma_{|g_1|}^{\text{sgn}(g_1)} \dots \sigma_{|g_\ell|}^{\text{sgn}(g_\ell)}$ with $1 \leq |g_j| < m$. The permutation obtained by exchanging adjacent strands is checked to be one cycle. Thus the input describes a knot, not a general link. For a one-component closure, $\ell \geq m - 1$: every adjacent gap in the strand path must be crossed at least once for the permutation to be transitive. An empty word on one strand is the crossing-free unknot.

A symbol operation acts on a generator, a free-product token, or a logarithmic-size index. A conservative bit interpretation charges $O(\log(\ell + m + 2))$ for such an operation. We state separately the larger-integer costs of the matrix method. The bounds do not apply to a binary exponent or straight-line program without charging for its expansion. No succinct-input assumption is hidden in ℓ .

The direct word API has the strongest stated specialized bound. Conversion to a PD code and its existing validation are separate preprocessing in the public **Diagram** API. Keeping braid provenance allows the public recognizer to use the word route after parsing; it does not make all existing parser operations constant time.

2 The relevant state of the art

There are two different reasons to improve this implementation: obtain better exact decisions now, and develop a route to a stronger general complexity theorem. They should not be conflated.

Lackenby’s 2021 announcement describes a quasi-polynomial unknot algorithm [2]. The July 2026 preprint supplies a hierarchy algorithm and additional constructive and certificate results, but its Section 9 separates iteration counts from the cost of its individual steps [3]. Our project has not implemented the announced general bound. We also do not infer from the absence of a cost proof in that preprint that the announcement is false.

The January 2026 preprint of Kelomäki and Schütz proves that the usual scanning and divide-and-conquer Khovanov algorithms can take exponential time even on positive or alternating three-braids. Their specialized three-braid computation avoids that obstruction using structural formulas [4]. This supports a change of representation or decision problem, not a further claim that width alone controls the existing explicit complex.

Our implementation asks only whether the knot is trivial. It need not compute every homology group, and it need not construct a minimal chain complex whose number of basis elements can itself be exponential. The three-braid route is a particularly clean example of this separation: an exact decision reduces to a short group-theoretic conjugacy test.

3 A complete three-braid classifier by free-product reduction

Write $a = \sigma_1$, $b = \sigma_2$, and use the standard presentation

$$B_3 = \langle a, b \mid aba = bab \rangle.$$

Put

$$x = aba, \quad y = ab, \quad z = x^2 = y^3 = (ab)^3.$$

Then

$$B_3 = \langle x, y \mid x^2 = y^3 \rangle, \quad Z(B_3) = \langle z \rangle, \quad B_3 / \langle z \rangle \cong C_2 * C_3.$$

These familiar presentations can also be checked by substituting $a = y^{-1}x$, $b = x^{-1}y^2$ in both directions. Let $e : B_3 \rightarrow \mathbb{Z}$ be the exponent-sum homomorphism; $e(z) = 6$.

3.1 The topological input

We use the following consequence of the closed-three-braid classification [5]; Datta states the unknot fiber explicitly [8, Theorem 4.1(i)].

Theorem 3.1 (Classical three-braid unknot classification). *The braids in B_3 with unknot closure form precisely the conjugacy classes represented by*

$$ab, \quad (ab)^{-1}, \quad ab^{-1}.$$

The representatives close to unknots: their two crossings are successive positive or negative stabilizations of the one-strand unknot, with mixed signs in the last case. The nontrivial content of the cited theorem is necessity. We use it as an explicit topological input, rather than silently assuming that every unknotted braid destabilizes monotonically.

Lemma 3.2 (The exponent sum fixes the central lift). *If two braids have conjugate images in $C_2 * C_3$ and equal exponent sums, then they are conjugate in B_3 .*

Proof. Lift a quotient conjugator to $g \in B_3$. If the two braids are β, γ , then $g\beta g^{-1} = \gamma z^k$ for some $k \in \mathbb{Z}$. Taking exponent sums gives $e(\beta) = e(\gamma) + 6k$. Equality forces $k = 0$. $\square$

3.2 An explicit constant-alphabet algorithm

In the quotient, the four input letters become

$$a \mapsto y^2x, \quad a^{-1} \mapsto xy, \quad b \mapsto xy^2, \quad b^{-1} \mapsto yx. \quad (1)$$

These are quotient identities. Replacing y^{-1} by y^2 in B_3 itself would lose a central power; that is why the exponent sum must be retained.

Represent x, y, y^2 by three tokens. Scan at most 2ℓ tokens into a stack. Adjacent x 's cancel. Adjacent powers of y combine by addition modulo three, with exponent zero deleted. If the incoming token belongs to a different free factor from the top token, push it. The resulting word alternates between the factors.

For cyclic reduction, use a deque. If the first and last syllables belong to the same factor, write the word uvw , with those endpoint syllables u, w . Conjugation by u^{-1} changes it into $v(wu)$. Delete the endpoints and append their nonidentity product. This decreases the number of syllables. Repeat until the length is at most one or the endpoint factors differ. No search over conjugators is necessary.

Lemma 3.3 (The required free-product conjugacy tests). *In $C_2 * C_3$, an element is conjugate to y or y^2 exactly when cyclic reduction yields that one syllable. It is conjugate to y^2xyx exactly when cyclic reduction has four alternating syllables, with one y and one y^2 .*

Proof. A reduced free-product word is unique. The usual conjugacy argument can be seen directly by cancelling a reduced conjugator against the two ends of a cyclically reduced word. For length at least two, a conjugate that is again cyclically reduced can only cyclically permute its syllables. For a word of length one, conjugacy stays within its factor; the factors here are abelian. The possible four-syllable words in the statement are exactly the cyclic rotations of y^2xyx . This proves both claims. $\square$

Theorem 3.4 (Complete symbolic criterion). *For a validated three-braid word β , let w be the cyclically reduced quotient word obtained above. Its closure is the unknot if and only if one of the following holds:*

$e(\beta)$	w	Representative in B_3
2	y	ab
-2	y^2	$(ab)^{-1}$
0	length four, with y and y^2 once each	ab^{-1}

Proof. The images of the three representatives are respectively y , y^2 , and y^2xyx , and their exponent sums are 2, -2, 0. Necessity follows from Theorem 3.1. For sufficiency, Lemma 3.3 gives a quotient conjugacy to the corresponding representative, and Lemma 3.2 lifts it to a braid conjugacy. Taking closures preserves its knot type. $\square$

Theorem 3.5 (Cost of the direct word backend). *The symbolic criterion, including word and component validation for at most three strands, uses $O(\ell)$ symbol operations and $O(\ell)$ stored tokens. A conservative logarithmic-address bit bound is $O(\ell \log(\ell + 2))$ time and $O(\ell \log(\ell + 2))$ bits of storage.*

Proof. Each input generator produces exactly two quotient tokens. A token is pushed at most once, and every stack cancellation removes a previously stored token. Thus the total number of stack updates is linear. Each cyclic-end operation decreases the deque length, so their total is also linear. The exponent sum has $O(\log(\ell + 2))$ bits. On at most three strands, component validation maintains a permutation of constant size. Checking the accepted word types is constant time after reduction, aside from emitting an optional word witness of length $O(\ell)$. Charging logarithmic index and counter costs gives the stated conservative bit bounds. $\square$

This is a bound for *decision from a supplied braid word*. It neither constructs a three-braid presentation of an arbitrary PD diagram nor computes the entire Khovanov homology of the closure.

4 An independent integer-matrix criterion

An independently structured implementation is valuable here: a sign error in the word-to-quotient map would otherwise affect both the mathematical description and its tests. Define

$$L = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad R = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad \rho(a) = L, \quad \rho(b) = R^{-1}.$$

The braid relation holds. This is reduced Burau at $t = -1$. It is onto $\mathrm{SL}_2(\mathbb{Z})$, with kernel $\langle z^2 \rangle$; $e(z^2) = 12$.

Lemma 4.1 (Small-trace conjugacy). *There are two conjugacy classes in $\mathrm{SL}_2(\mathbb{Z})$ with trace 1, represented by $\rho(ab)$ and its inverse. There is one conjugacy class with trace 3, represented by $LR = \rho(ab^{-1})$.*

Proof. For trace 1, Cayley–Hamilton gives $M^3 = -I$. Its image in $\mathrm{PSL}_2(\mathbb{Z}) = C_2 * C_3$ has order three. A finite-order element of a free product is conjugate into a factor: otherwise its cyclically reduced word has length at least two and all its positive powers remain reduced and grow. The two order-three classes are therefore the two nonidentity elements of C_3 . Choosing the lift with trace 1 fixes the sign and yields the two stated classes.

For trace 3, use

$$S = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \quad U = SL.$$

In the projective quotient, S has order two and U order three. We have $SU = -L$ and $SU^{-1} = R$. A cyclically reduced infinite-order word is an alternating product of S and $U^{\pm 1}$. Consequently a lift is conjugate, up to sign, to a positive word in L, R . A word in only one letter has trace 2. If both occur, rotate the cyclic word to begin with $L^r R^s$, $r, s \geq 1$. This prefix has trace $2 + rs$ and strictly positive entries. For a strictly positive matrix X ,

$$\mathrm{tr}(XL) = \mathrm{tr}(X) + X_{21}, \quad \mathrm{tr}(XR) = \mathrm{tr}(X) + X_{12}.$$

Every additional letter strictly increases the trace. Trace 3 therefore forces exactly LR , up to cyclic rotation. The negative lift has negative trace, so trace 3 selects the positive one. A projective conjugator lifts to an integral determinant-one conjugator, completing the proof. $\square$

Theorem 4.2 (Exact trace–exponent criterion). *For a validated three-braid knot,*

$$\widehat{\beta} \text{ is the unknot} \iff (e(\beta), \mathrm{tr} \rho(\beta)) \in \{(2, 1), (-2, 1), (0, 3)\}. \quad (2)$$

Proof. Necessity follows by evaluating the three classes of Theorem 3.1. For sufficiency, Lemma 4.1 gives a matrix conjugacy to the appropriate representative. For the trace-one alternatives, the two classes have exponent-sum residues 2 and -2 modulo 12; the residue is well-defined because the kernel has exponent 12. The specified exponent therefore selects the correct class. Lift the matrix conjugator to B_3 . The residual difference is a power of z^2 , and equality of exponent sums forces that power to be zero. $\square$

4.1 The determinant interpretation and a useful trap

For a three-braid knot, the Alexander–Burau determinant identity gives

$$\det(\widehat{\beta}) = |\det(I - \rho(\beta))| = |2 - \mathrm{tr} \rho(\beta)|. \quad (3)$$

This provides a further independent test against the existing exact Alexander implementation. The trace–exponent specialization has historical precedents in determinant-one closed-three-braid classifications [6, 8]; it is not a new classification claim.

The exponent is indispensable. The torus knot represented by $(ab)^5$ has trace 1 and determinant 1, but exponent sum 10. More generally, the words $(ab)^{6k+1}$ have the same quotient class and determinant as ab , while their exponent sums are $12k + 2$. A classifier that keeps only the quotient or determinant would give incorrect positive answers on this family.

The figure-eight word $(ab^{-1})^2$ has exponent zero, matrix $(LR)^2$ of trace 7, and determinant 5. Its quotient cyclic word has eight syllables, so both implemented criteria reject it.

4.2 Bit complexity and serialization

Writing a current matrix as $\begin{pmatrix} u & v \\ w & t \end{pmatrix}$, right multiplication by the four generators changes one column by adding or subtracting the other. For example, multiplication by a gives

$$(u, v, w, t) \mapsto (u, u + v, w, w + t),$$

and multiplication by b gives

$$(u, v, w, t) \mapsto (u - v, v, w - t, t).$$

After j letters, the absolute entry sum is at most $4 \cdot 2^j$. Every entry therefore has $O(j + 1)$ bits. Summing the costs of the constant number of additions per letter yields $O(\ell^2)$ elementary bit operations and $O(\ell)$ bits of matrix storage. The input word and witness require additional storage if retained.

The implementation encodes large matrix entries in signed hexadecimal when producing JSON. This avoids decimal-conversion limits and avoids mistaking a serialization failure for a mathematical failure. The free-product backend is the default; the matrix backend supplies independent verification and useful arithmetic witnesses.

5 Extending the useful input class by certified preprocessing

5.1 A linear braid-writhe obstruction

Bennequin’s theorem implies that an unknotted closed m -braid has $|e(\beta)| < m$, equivalently $|e(\beta)| \leq m - 1$ [9, pp. 100–101]. Hence

$$|e(\beta)| > m - 1 \implies \widehat{\beta} \text{ is nontrivial.}$$

This is an exact one-sided test for every strand number. It never promotes equality to an unknot verdict. It is particularly cheap on long positive or negative braid words and can avoid the general Alexander matrix entirely.

For one strand, the valid empty word is trivial. For two strands, $B_2 \cong \mathbb{Z}$, and a knot closure is the unknot exactly when the exponent sum is 1 or -1 . Together with Theorem 3.4, this completes the direct route for at most three strands.

5.2 Endpoint singleton destabilizations

Lemma 5.1 (A checked Markov step). *Suppose a word on $m > 3$ strands contains exactly one occurrence of a generator with absolute index $m - 1$. Delete that occurrence after a cyclic rotation placing it last. The remaining word is an $(m - 1)$ -braid with isotopic closure. The corresponding statement holds for an absolute-index-one singleton, after deleting the leftmost strand and decreasing all remaining absolute generator indices by one.*

Proof. If the original word is $u\sigma_{m-1}^\varepsilon v$, the absence of other endpoint generators gives $u, v \in B_{m-1}$. Cyclic conjugation gives $vu\sigma_{m-1}^\varepsilon$. A Markov destabilization removes the last crossing and last strand. For the left endpoint, conjugation by the half twist reverses the strand indices, reducing to the right-endpoint case; reversing the remaining indices back produces the stated renumbering. Both crossing signs are allowed. $\square$

The routine first freely reduces and cyclically freely reduces the Artin word. These operations are braid equalities or conjugations. It then searches only for the two sufficient endpoint conditions. After each successful destabilization it repeats, stopping at three strands or when no such step is available.

The certificate records the current strand count and reduced word length, the chosen endpoint, the occurrence position and signed generator, and the final word. Its verifier recomputes free reductions, checks that the chosen endpoint really occurs once, performs the chosen rotation and deletion, and checks the final result. The verifier does not run the search that selected the endpoint.

Proposition 5.2 (Certified descent bound). *For an explicit valid word of length ℓ on m strands, the implemented singleton descent uses $O(m\ell)$ word operations, never increases length, and performs at most $\max\{m - 3, 0\}$ destabilizations. Its certificate can be replayed within the same bound. If it reaches three or fewer strands, the subsequent classifier gives a complete decision for the original closure.*

Proof. Each pass performs linear free/cyclic reduction and counts endpoint occurrences. Each successful pass reduces the strand count by one. There is at most one final unsuccessful pass. Rotation and renumbering cost $O(\ell)$; all later words have length at most the original. The same bound holds for replay, which checks rather than searches for each chosen occurrence. Soundness follows by composing Lemma 5.1 and the free reductions, and then applying the complete low-strand criterion. $\square$

Since $m \leq \ell + 1$ for a one-component input, the routine has a polynomial worst-case bound. This extends the certified family beyond inputs initially declared to have three strands, without claiming an algorithm for minimal braid index.

5.3 A necessary negative control: Morton’s example

Morton’s four-braid unknot [10], in the word convention displayed by Birman–Brendle [11, Example 3.1], is

$$X = \sigma_3^{-2} \sigma_2 \sigma_3^{-1} \sigma_2 \sigma_1^3 \sigma_2^{-1} \sigma_1 \sigma_2^{-1}.$$

Its explicit word is

$$[-3, -3, 2, -3, 2, 1, 1, 1, -2, 1, -2].$$

The exponent sum is 1, so the writhe test is inconclusive. The left and right endpoint generators occur four and three times respectively. The singleton routine therefore stops immediately. The new preprocessing returns no verdict and the existing general recognizer supplies the unknot answer. This example is in the tests.

More generally, a failure to descend says nothing about whether a closure is knotted or whether a different representative has fewer strands. Exchange moves and other transformations can be necessary.

5.4 A closure certificate is not a local tangle replacement

An important boundary of applicability is easy to miss. A three-braid whose *closure* is an unknot need not be the identity braid. For example ab has unknot closure and exponent two, so it is not the identity. Replacing occurrences of it by the identity inside another braid can change the resulting knot.

The new classifier is used on a whole verified closed braid, or after whole-closure-preserving Markov steps. It cannot be used to erase an arbitrary three-strand subword of a larger tangle. Likewise, the existing connected-sum shortcut is justified by an actual splitting sphere, not merely by recognizing an auxiliary closure.

6 Integration and exact semantics

The parser previously converted braid input to a normalized PD code and discarded the word. The updated parser retains immutable validated braid provenance. PD equality and hashing continue to refer to the same diagram data. The new decision route is invoked before geometric simplification discards that provenance.

The implementation validates provenance rather than accepting a user assertion that an unrelated PD has a three-braid representation. Serialization, restoration, and mirror handling are tested. The direct word API is also available for large explicit words when no PD construction is needed.

The resulting order of work is:

1. Validate a one-component input and any retained braid provenance.
2. When braid provenance is available, apply the writhe obstruction and the complete route for at most three strands.

3. For unresolved higher-strand inputs, first run the existing cheap Reidemeister I/II and descending checks, then optional certified endpoint descent on the original validated source word. The original closure and the simplified diagram are isotopic, so this certificate remains valid.
4. If these do not finish, use the existing polynomial, factorization, Reidemeister III, and Khovanov pipeline.

Delaying higher-strand descent avoids repeated word passes on easy kink chains already handled by the cheap geometric checks. The raw word API still offers descent directly, since it does not construct a PD diagram. Each added early decision is justified independently of the fallback. The fallback is unchanged mathematically. Its complete decision theorem is reduced Khovanov unknot detection [14]: rank one over $\mathbb{F}_2$ occurs exactly for the unknot. Indeed rational reduced rank is at most the rank over $\mathbb{F}_2$ by universal coefficients, and rational rank one detects the unknot; the unknot itself has rank one over both fields.

A resource limit remains a third outcome, UNKNOWN. It is not a negative answer. Backend options allow the new route to be disabled when testing or benchmarking the general scanner. Resource-limit regression tests that intentionally force that scanner explicitly turn off the new route; otherwise a valid early certificate would correctly finish before the intended limit.

6.1 Code delivered

Component	Purpose
Braid backend module	Direct symbolic and matrix decisions, writhe witness, component checks and statistics
<code>braid_reduction.py</code>	Elementary Markov descent and independent certificate replay
<code>residue.py</code>	Non-mutating scalar-residue multiplicity diagnostic
Parser and recognizer integration	Retained provenance, options, serialization and exact fallback behavior
Tests and experiment scripts	Exhaustive comparisons, independent oracles, paired timings and intrinsic-profile checks

The mathematical distinction between decision and full invariant output is retained in the API. A braid certificate proves a verdict; it is not a request to fabricate uncomputed Khovanov groups. The existing full-rank APIs continue to mean full-rank computations.

7 Intrinsic multiplicities of a partial scanning complex

The second main result addresses the general scanner rather than only braid input. We work over $\mathbb{F}_2$, as the implementation does. Fix one prefix, its boundary, and its matching objects. A complex is a finite direct sum of matchings in each homological degree, with differential entries in the dotted cobordism algebra.

For a boundary with $2b$ points and two matchings a, c , let $k(a, c)$ be the number of circles in their alternating union. A morphism has a basis indexed by dot subsets on these circles. In particular

$$\text{End}(a) \cong \mathbb{F}_2[x_1, \dots, x_b] / (x_1^2, \dots, x_b^2).$$

Its units are exactly the polynomials with constant coefficient one.

7.1 The scalar residue functor

Define q on a morphism by discarding it when the two matching objects differ, and by taking its undotted coefficient when they agree. The target is a category with one independent scalar identity for each matching.

Lemma 7.1 (Residue respects composition). *For the matching cobordism algebra used by the scanner, q is additive, preserves identities, and satisfies $q(gf) = q(g)q(f)$.*

Proof. Additivity and identities are immediate. A composition with distinct initial and final matchings is discarded on both sides. For $a \rightarrow a \rightarrow a$, composition is multiplication in the displayed square-zero polynomial algebra, and constant terms multiply.

It remains to consider $a \rightarrow c \rightarrow a$ with $c \neq a$. Then $k(a, c) < b$: having b alternating-union circles forces every arc to agree. In the gluing surface, every connected component contains an input circle of the left morphism. Thus there are at most $k(a, c)$ components, whereas the output consists of b circles. An undotted constant output can arise only when each surface component has exactly one output circle and no dot: a genus-zero component with two or more output circles contributes at least one dot, and positive-genus components vanish over $\mathbb{F}_2$. A component with no output cannot increase the number of components available for output. A constant output would require at least b components, contradicting $k(a, c) < b$. Therefore its undotted coefficient vanishes. $\square$

The proof uses the algebra of the fixed prefix and does not require that all boundary points lie on one disk boundary. For disk tangles there is also the shorter positive-grading proof in Section 8.

Let $N_{h,a}$ be the number of copies of matching a in cube degree h , and let $D_{h,a}$ be the scalar differential matrix from (h, a) to $(h+1, a)$ after applying q . By the lemma, $D_{h+1,a}D_{h,a} = 0$. Set

$$\mu_{h,a}(C) = N_{h,a} - \text{rank } D_{h,a} - \text{rank } D_{h-1,a}, \quad \mu(C) = \sum_{h,a} \mu_{h,a}(C). \quad (4)$$

Theorem 7.2 (Canonical post-cancellation multiplicities). *Any exhaustive sequence of invertible-entry cancellations in C leaves exactly $\mu_{h,a}(C)$ copies of matching a in degree h . These numbers depend only on the chain-homotopy type with its fixed homological normalization.*

Proof. Categorical Gaussian cancellation is a chain-homotopy equivalence [13]. Applying an additive functor preserves chain maps, homotopies, and their equations, so applying q to a cancellation preserves the homology of the scalar quotient complex.

Once no invertible entry remains, every same-matching entry has constant term zero and every different-matching entry is killed by q . The residue differential of the remaining complex is therefore zero. Its number of copies in each (h, a) is its residue homology dimension. That dimension agrees with the original complex and is exactly (4). The same argument proves invariance under any chain-homotopy equivalence with the stated normalization. $\square$

Khovanov’s minimal-core results supply the general algebraic background [12, Section 5.1]. Equation (4) is the concrete diagnostic appropriate to this code.

7.2 What pivot optimization can change

The theorem gives a sharp interpretation of pivot experiments. At the *same completed prefix*, every exhaustive pivot strategy has the same remaining object multiplicities. A better strategy can still reduce:

- the number of compositions and the size of intermediate differential fill-in;

- temporary storage during a cancellation stage;
- bookkeeping and cache overhead.

It cannot reduce the final minimal object count at that prefix.

Changing scan order changes the partial tangles, and therefore can change their intrinsic multiplicities. Delaying cancellation across several crossings also changes the sequence of completed prefixes being materialized. These remain meaningful directions; the theorem does not say that every ordering has the same complexity.

The code reports cube degree. Reidemeister changes can shift the usual normalized homological degree, so degree-by-degree comparisons across different presentations must align conventions. Our pivot comparisons use identical prefixes and require no such shift.

7.3 Implemented diagnostic and its limitation

The diagnostic builds scalar blocks, computes their $\mathbb{F}_2$ -ranks with bit-packed elimination, and returns the nonzero $\mu_{h,a}$. It accepts the optimized scanner and the set- and bit-based generic scanners. It optionally checks the scalar equation $d^2 = 0$, verifies degree consistency, and does not mutate the complex.

It is deliberately not an extra default pass before every elimination: that would duplicate work unless the profile is being used for an audit or a different batching algorithm. Its purpose in this bundle is to verify an invariant, make measurements interpretable, and provide a basis for subsequent algorithms.

Crucially, the multiplicity list alone is not a replacement for the partial complex. Positive-degree differential entries survive in the minimal complex. Later crossings can interact with them. Dropping those entries and retaining only the list would not be an exact Khovanov algorithm.

8 Finite batch cancellation through the radical

This section develops a concrete next algebraic primitive, with a proof and cost bound. It is not described as an implemented production speedup.

Assume first that the prefix is represented as a disk tangle with $2b$ boundary points. The standard arc algebra is

$$A_b = \bigoplus_{a,c \in \mathcal{M}_b} \text{Hom}(a, c),$$

where $\mathcal{M}_b$ is the set of crossingless disk matchings. A basis element with d dots has intrinsic algebra degree

$$\deg_{\text{alg}} = b - k(a, c) + 2d. \quad (5)$$

This is the positive grading from the arc-algebra construction [12, Section 2.4]; it is distinct from the homological degree and from presentation shifts in quantum degree.

Lemma 8.1 (A bounded radical filtration). *The algebra A_b is graded in degrees $0, \dots, 2b$. Its degree-zero part is $\prod_{a \in \mathcal{M}_b} \mathbb{F}_2 e_a$. Its positive-degree ideal J_b satisfies $J_b^{2b+1} = 0$; projection onto degree zero is the residue functor.*

Proof. Since $k \leq b$ and $0 \leq d \leq k$, (5) lies between zero and $2b$. It is zero only for $a = c$ and the undotted identity. Composition preserves degree. This can also be checked from the Frobenius operations: every split adds one dot in each nonzero summand, whereas a merge adds none; counting the b saddles in a composition gives additivity of (5). Thus positive degree is an ideal, and a product

of $2b + 1$ positive terms must vanish. The quotient is the displayed product of fields. Over $\mathbb{F}_2$, this ideal is also the Jacobson radical. $\square$

Theorem 8.2 (Finite simultaneous reduction). *Let $d = d_0 + \delta$, where d_0 consists of the scalar equal-matching coefficients of a partial complex. Choose a scalar contraction of (C, d_0) onto its zero-differential homology object H :*

$$pi = 1_H, \quad d_0h + hd_0 = 1_C + ip, \quad h^2 = hi = ph = 0, \quad d_0i = pd_0 = 0.$$

All entries of i, p, h can be scalar matching identities. Define

$$\mathcal{T} = \delta \sum_{j=0}^{2b} (h\delta)^j, \quad d_H = p\mathcal{T}i.$$

Then (H, d_H) is chain-homotopy equivalent to (C, d) , has exactly the multiplicities $\mu_{h,a}(C)$, and is fully unit-reduced.

Proof. The residue functor implies $d_0^2 = 0$. Scalar Gaussian elimination in each matching block supplies the stated special contraction. The entries of δ lie in J_b , while h has algebra degree zero. Hence $(h\delta)^{2b+1} = 0$, so the displayed sum is the exact finite inverse of $1 + h\delta$, multiplied on the left by δ . There is no convergence or numerical approximation issue.

For completeness, put

$$i' = i + h\mathcal{T}i, \quad p' = p + p\mathcal{T}h, \quad h' = h + h\mathcal{T}h.$$

Over $\mathbb{F}_2$, the finite perturbation identities are

$$\mathcal{T} = \delta + \delta h\mathcal{T} = \delta + \mathcal{T}h\delta, \quad d_0\mathcal{T} + \mathcal{T}d_0 = \mathcal{T}ip\mathcal{T}.$$

The second follows by inserting $d_0\delta + \delta d_0 = \delta^2$ and $d_0h + hd_0 = 1 + ip$ in the finite series. They imply

$$d_H^2 = 0, \quad di' = i'd_H, \quad p'd = d_Hp', \quad p'i' = 1_H, \quad dh' + h'd = 1_C + i'p'.$$

These are the required chain-homotopy equations. They are the homological perturbation formulas specialized to the nilpotent ideal [15]. Every term of d_H contains a δ , so its entries have positive algebra degree. It therefore has no cancellable unit. $\square$

For $b = 0$, the radical is zero and this reduces to ordinary homology. For $b > 0$, computing only H is insufficient: the transferred d_H must be retained. The nilpotence index bounds the number of matrix products, not the size of their coefficient matrices.

8.1 A conservative explicit arithmetic bound

Let M be the number of objects before a stage is reduced and $D = \dim_{\mathbb{F}_2} A_b$. If ω is an admissible exponent for matrix multiplication over $\mathbb{F}_2$, a direct dense realization of Theorem 8.2 uses

$$O((b+1)D^3M^\omega)$$

field operations, in addition to building the finite multiplication table. Expand entries in an algebra basis: one algebra-valued matrix product is at most D^3 coefficient-matrix products. There are $O(b+1)$ such products in the formula. Scalar rank decomposition supplies a contraction within $O(M^\omega)$ field operations.

For the disk algebra,

$$D = \sum_{a,c} 2^{k(a,c)} \leq \text{Cat}(b)^2 2^b = 2^{O(b)}.$$

Thus the dense bound is $2^{O(b)} M^\omega$. This can be poor on sparse small instances; it is an asymptotic implementation target, not a claim that dense multiplication will beat the current sparse scanner in the supplied measurements.

The disk hypothesis is real. For an arbitrary multi-boundary prefix, one must specify its actual matching algebra or prove a disk description. A safe count of abstract matchings on $2b$ points is $(2b-1)!!$, not automatically $\text{Cat}(b)$. Planarity of the full knot diagram by itself does not supply the missing boundary description.

9 What a width–multiplicity bound really proves

Let a specified crossing order have at most $2b$ active boundary points, and let

$$\mu = \max\{1, \mu(C_i) : 0 \leq i \leq n\}$$

be its largest intrinsic minimal multiplicity. After a crossing is added to a reduced prefix, there are two smoothings, and each can create at most two closed circles. Delooping therefore creates at most 8μ objects before the next reduction.

Theorem 9.1 (A safe scalar profile bound). *There is an exact scalar-cancellation realization using the same matching representation whose cost, for the specified scan, is*

$$\tilde{O}(n 2^{O(b)} \mu^3).$$

In particular,

$$b + \log(\mu + 1) = O(\log^2(n + 2))$$

is sufficient for a quasi-polynomial bound of the form $n^{O(\log n)}$.

Proof. At one stage there are $M = O(\mu)$ objects. Locate a unit by scanning at most M^2 entries. A cancellation removes two objects and changes at most M^2 differential entries. There are at most $M/2$ cancellations. Each morphism has at most 2^b dot monomials and composition can be evaluated by an explicit $2^{O(b)} \text{poly}(b)$ procedure. The entry scans, compositions and bit-packed additions therefore cost $2^{O(b)} M^3$, up to logarithmic bookkeeping. Transfer at a crossing costs no more at this scale. Sum over n stages. Taking logarithms of the bound proves the final assertion. $\square$

The theorem describes a definite available scalar algorithm. It is not an unproved assertion that every stale-priority queue or racing variant in the current code has exactly this overhead bound. The measured intrinsic profile remains meaningful for those variants.

For disk frontiers, the batch formula gives $n 2^{O(b)} \mu^\omega$, with the same conceptual limitation. In both cases, the problem is establishing or exploiting a useful bound on μ . A measured small profile for a benchmark is not a proof of such a bound on all knots.

9.1 An explicit fixed-width obstruction

For $r \geq 1$, consider the alternating three-braid word

$$\beta_r = (ab^{-1})^r.$$

For $3 \nmid r$, its closure is a knot. The crossing order inherited from the braid word has at most six frontier endpoints: at most three at the moving level and three associated with the closure. Set $\lambda_{\pm} = (3 \pm \sqrt{5})/2$. Since LR has eigenvalues $\lambda_{\pm}$,

$$\det(\widehat{\beta_r}) = \text{tr}((LR)^r) - 2 = \lambda_+^r + \lambda_-^r - 2.$$

The reduced Khovanov Euler characteristic is the normalized Jones polynomial, with the usual square-root change of variable. Its graded Euler characteristic is unchanged by passing to $\mathbb{F}_2$. In a convention where the Jones variable is $t = q^2$, evaluate at $q = i$; then $|V_K(-1)| = \det(K)$, and every monomial has absolute value one. The triangle inequality therefore bounds the determinant by the total reduced Betti number, giving

$$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(\widehat{\beta_r}) \geq \det(\widehat{\beta_r}).$$

Consequently the size of an explicit minimal complex grows exponentially in r , despite the fixed number of braid strands. This elementary family illustrates the representation obstruction analyzed more broadly in [4].

This is a statement about materializing the homology or its minimal complex. The baseline’s determinant filter already rejects this family cheaply. We do not use a deliberately disabled filter to claim an end-to-end speedup of the new pipeline. The relevant lesson is that an explicit-basis fallback cannot acquire a general polynomial width bound solely from improved cancellation bookkeeping.

9.2 Decision, output size, and compressed multiplicity

An exponentially large rank can be stored as an integer with linearly many bits. An explicit basis with that many elements cannot. The existing connected-sum shortcut already uses this distinction. The three-braid classifier takes it further by computing only a complete decision invariant for its class.

For general tangles the difficult information lies in the differential, not only in the counts of objects. A useful compressed representation must support exact gluing, composition, and rank or equivalence tests without expanding the multiplicities. The residue theorem says what survives scalar cancellation; it does not make those operations free.

10 Experiments and verification

All empirical claims below concern completed runs on the supplied implementation and pinned baseline. The experiment artifacts record the interpreter, platform, options, exact inputs or seeds, operation counts, and raw paired timings. Wall-clock results are evidence about these workloads, not proofs of complexity.

10.1 Independent correctness checks

The pinned baseline passes 41 unit tests; the delivered implementation passes 59. These test counts include the following exhaustive comparisons, not merely a handful of selected examples.

The new three-braid tests exhaust all 46,376 one-component words of lengths two, four, six and eight over $\{a, a^{-1}, b, b^{-1}\}$, comparing the free-product and matrix criteria. For all 2,856 such words through length six, an independent dense reduced-Khovanov cube is used as another decision oracle.

Additional checks compare the determinant formula against the existing exact Alexander polynomial on 120 seeded words; test braid relations and conjugation in 200 seeded contexts; stress long words and large matrix serialization; and exercise provenance, mirrors, invalid input, resource limits and

fallback behavior. These compare different mathematical computations rather than only duplicate implementations of one predicate.

The Markov tests compare unreduced input and reduced output through the existing Khovanov engine for both endpoint choices and both crossing signs. They test repeated stabilizations under conjugation and reject tampered certificates. Morton’s example is a mandatory inconclusive preprocessing case.

For multiplicities, every tested crossing prefix is transferred without elimination, its scalar-residue Betti numbers are computed, and then the actual categorical cancellations run. The predicted and observed matching-by-degree profiles must agree, including across the optimized, generic bit-based, and generic set-based engines with distinct pivot rules. An additional uncanceled closed complex is checked against the existing full linear-homology routine.

10.2 Paired performance measurements

The benchmark separates direct word processing from public pipeline calls on parsed diagrams. It also separates the baseline default pipeline from a named ablation that disables Reidemeister III search. The latter is useful for exposing a scanner workload, but is not substituted for the default baseline comparison.

The final run used CPython 3.12.14 on x86_64 Linux, with 15 paired rounds per case and a rotating three-arm order: baseline, new, and a second baseline call. Each implementation had one warm-up. The timed region includes fresh PD construction and recognition; it excludes process startup, imports, and JSON decoding. All published samples finished with the expected exact verdict.

The comparison rule uses an order-statistic interval with at least 95% coverage for the median new/old ratio under independent sampling. It labels a case faster only when the interval lies below one and the median ratio is below the first quartile of the paired baseline-control ratios. The slower rule is symmetric. These are exploratory workload comparisons with a local A/A noise control; they are not simultaneous confidence guarantees across the whole corpus.

Twenty-one of 28 cases satisfy the faster rule, seven support no timing claim, and none satisfy the slower rule. This is a statement about these completed measurements, not a proof that regressions are impossible. The default-pipeline speedup ranges are 3.97–7.04 for positive three-braids, 2.39–3.88 for alternating three-braids, and 2.44–5.29 for the five scrambled three-braid unknots. The larger 10.72–51.05 range belongs to the explicitly labeled R3-disabled ablation.

Table 1: All paired pipeline measurements. Times are separate medians in milliseconds; speedup is the median of paired old/new ratios, which need not equal the quotient of the displayed medians. “None” means no timing claim under the stated rule.

Case	n	Old ms	New ms	Speedup	Claim
Positive three-braid, $r = 5$	10	0.351	0.087	4.09×	Faster
Positive three-braid, $r = 11$	22	0.630	0.156	3.97×	Faster
Positive three-braid, $r = 23$	46	1.559	0.272	4.40×	Faster
Positive three-braid, $r = 47$	94	3.966	0.575	7.04×	Faster
Positive three-braid, $r = 95$	190	8.705	1.452	5.67×	Faster
Alternating three-braid, $r = 5$	10	0.210	0.085	2.39×	Faster
Alternating three-braid, $r = 11$	22	0.448	0.158	2.91×	Faster
Alternating three-braid, $r = 23$	46	2.040	0.424	3.88×	Faster
Alternating three-braid, $r = 47$	94	2.752	0.871	2.52×	Faster
Alternating three-braid, $r = 95$	190	7.481	2.244	2.72×	Faster

Continued on next page

Case	n	Old ms	New ms	Speedup	Claim
Scrambled unknot, seed 0	190	6.059	2.221	2.44×	Faster
Scrambled unknot, seed 0 (R3 off)	190	17.365	1.731	10.72×	Faster
Scrambled unknot, seed 4	158	6.295	1.197	5.12×	Faster
Scrambled unknot, seed 4 (R3 off)	158	31.022	1.502	21.36×	Faster
Scrambled unknot, seed 5	202	10.292	2.411	4.42×	Faster
Scrambled unknot, seed 5 (R3 off)	202	80.592	1.563	51.05×	Faster
Scrambled unknot, seed 7	198	6.787	1.394	4.36×	Faster
Scrambled unknot, seed 7 (R3 off)	198	58.837	2.089	27.66×	Faster
Scrambled unknot, seed 14	182	6.815	1.259	5.29×	Faster
Scrambled unknot, seed 14 (R3 off)	182	19.499	1.185	14.83×	Faster
Morton’s four-braid unknot	11	2.818	2.913	1.03×	None
Stabilized unknot, 16 strands	15	0.120	0.133	0.93×	None
Stabilized unknot, 64 strands	63	0.474	0.544	0.81×	None
Stabilized unknot, 256 strands	255	2.838	2.206	1.20×	None
Conway	11	0.820	0.795	0.97×	None
Kinoshita–Terasaka	11	0.932	0.869	1.04×	None
Eight-crossing unknot (braid)	8	0.508	0.092	5.57×	Faster
Grid unknot	6	0.080	0.078	1.02×	None

Raw word timings are separate five-repetition experiments. At 128,000 letters the symbolic median is 0.112884 seconds and the matrix median is 0.626293 seconds; the latter’s largest final entry has 88,863 bits. At 1,024,000 letters the symbolic median is 1.044478 seconds, with exactly 2,048,000 quotient-token operations. At 1,000 letters, however, the matrix code is faster (0.000258 versus 0.000744 seconds). The asymptotically better symbolic method does not win every small case. Raw scaling samples do not use the three-arm comparison rule and are reported as descriptive timings.

For repeated calls, baseline and new calls are interleaved, and both orders occur. Ratios are computed from paired observations. Parsing and fresh-process startup are reported only when included by the named workload. The raw data, rather than a cross-machine historical table, are the authority for these ratios.

10.3 Deterministic complexity evidence

Operation counts test the linear word-processing bound independently of timer noise. Long conjugated words and words whose reduced quotient stays long exercise different stack and cyclic-reduction behavior. The matrix experiment records the largest final matrix-entry bit length and contrasts its larger-integer cost with constant-alphabet reduction.

A second experiment records intrinsic multiplicities, boundary size, cancellation counts and composition counts for identical prefix orders. It checks that pivot changes can alter work while preserving the canonical completed-prefix multiplicities. The data do not extrapolate small observed profiles into a universal bound.

The deterministic profile experiment includes 54 prefixes on six diagrams, checked in four scanner configurations: 216 pre/post-cancellation checks. Every predicted multiplicity and every completed profile agrees. The table compares composition calls in the two generic bit-based engines, so their counters have the same meaning; optimized identity shortcuts are not mixed into this comparison.

Diagram	Crossings	Peak μ	Min-fill calls	LIFO calls
Conway	11	66	770	1281
Kinoshita–Terasaka	11	66	671	1277
Figure-eight	4	10	15	18
Eight-crossing unknot	8	4	13	24
$T(3, 5)$	10	14	113	132
$(ab^{-1})^5$	10	242	1216	1707

For example, both Conway runs retain the same maximum of 66 objects, while their composition counts differ. This is precisely the distinction proved by Theorem 7.2.

10.4 Limits of the experimental evidence

The new complete route requires supplied or certified braid structure. A PD input that happens to represent a three-braid knot is not automatically routed there. Geometric discovery of the representation is outside these measurements.

The general algorithm still has an exponential fallback. The new code does not implement the radical batching formula, a compressed general Khovanov differential, or the missing geometric hierarchy engine. No measured speedup is used as evidence that these open tasks have already been completed.

11 The geometric route to the quasi-polynomial target

The 2026 hierarchy work is important because it identifies mathematical operations that a full implementation could represent explicitly. It should be used with the precise complexity distinction made in its Section 9 [3].

Write L for the maximum hierarchy length and g for the maximum pattern-complexity $\chi_p(S)$ of any decomposing surface appearing during the algorithm. Proposition 9.1 supplies an iteration bound of the form

$$L(g + 1)^L.$$

An iteration count is not a complete bit-complexity analysis. It must be combined with representation sizes, preparation costs, the exact cost of each operation, and its change to later states.

Proposition 11.1 (A noncircular complexity-transfer contract). *Suppose a hierarchy implementation on an n -crossing input has initialization cost $P(n)$, at most $L(n)(g(n) + 1)^{L(n)}$ iterations, and a proved uniform bit cost $Q(n)$ for an iteration, including all data updates and any necessary verification. Then*

$$T(n) \leq P(n) + L(n)(g(n) + 1)^{L(n)}Q(n).$$

In particular, if $P(n), Q(n) \leq n^{O(\log n)}$, $g(n) \leq n^{O(1)}$, and $L(n) = O(\log n)$, then $T(n) = n^{O(\log n)}$.

Proof. Sum the uniform cost over the proved number of iterations and add initialization. Under the stated bounds,

$$\log(L(g + 1)^L Q) = O(\log \log n) + O((\log n)^2) + O((\log n)^2).$$

Exponentiating gives the claim. □

Here $Q(n)$ must be proved using the evolving encoded state, not defined to be quasi-polynomial by assumption without an encoding argument. If $L = O(\log^2 n)$ and g remains polynomial, this particular product bound yields $n^{O(\log^2 n)}$, not the sharper $n^{O(\log n)}$ target. Both are often called quasi-polynomial; the exponents must be kept explicit.

The missing project obligations are concrete:

1. An encoded handle structure, boundary pattern, and hierarchical multi-surface with clear provenance and bit-size bounds.
2. Compressed cutting and gluing that do not expand large normal coordinates into that many physical pieces.
3. Constructive simplification, weak reduction, and the required geometric region operations, with exact topological contracts.
4. A decreasing potential and a depth or aggregate-work estimate strong enough for the target exponent.
5. A complete cost analysis for all intermediate state changes.

A polynomially checkable certificate is valuable but does not provide a polynomial-time certificate constructor. Likewise, an interval pairing or normal-coordinate primitive does not by itself supply every hierarchy transformation. These distinctions explain what a next implementation must prove rather than merely promise.

12 Research questions and proposed next work

The following questions are chosen for their effect on the actual remaining costs. Each includes a concrete success criterion.

12.1 Recover certified small-braid descriptions from PD inputs

Can a practical algorithm construct a three-braid representation, or a sequence of whole-closure-preserving moves leading to one, for a useful class of PD or grid inputs? The output should be a replayable certificate with a proved relation between certificate size and original input size.

A useful first result would recognize a nontrivial structural class in polynomial time, then invoke the backend proved here. Merely noting that a knot has braid index three does not provide the representation required by the algorithm.

12.2 Go beyond singleton endpoint reductions

Implement certified exchange moves, bounded bands of braid relations, or a controlled low-strand conjugacy search before attempting destabilization. Morton's four-braid is a small mandatory example, but success on it alone is not a general theorem.

A meaningful complexity result should bound total explored states or supply an independently decreasing potential. The search should retain its incomplete outcome and hand unhandled cases to the exact fallback.

12.3 A complete structural decision route for four-braids

The three-braid quotient has an exceptionally simple central extension. There is no justified instruction to reuse the same predicate on four strands. Can four-braid unknot recognition be reduced to finitely many structural normal forms or to a compressed invariant that is complete for the trivial knot?

This is more focused than computing every invariant, and complements the fixed-strand Khovanov-complexity questions in [4]. A first milestone is a complete subclass with explicit encodings and a cost proof, together with counterexamples to overgeneralized trace tests.

12.4 Use intrinsic multiplicity to choose scan orders

A frontier-width objective ignores the invariant measured by $\mu(C_i)$. Can inexpensive algebraic probes predict that invariant well enough to select better next crossings? Can one prove approximation or parameterized guarantees for an order minimizing $\max_i \mu(C_i)$, perhaps jointly with boundary size?

The residue diagnostic can label training or benchmark instances, but computing the exact profile after building the full stage does not itself give a cheap predictor. Tests should compare actual composition counts and peak memory, not only a surrogate score.

12.5 Implement and compare radical batching

Build the scalar contraction and transferred positive-degree differential of Theorem 8.2. Verify the full chain-homotopy equations on small real tangle complexes, then compare sparse, block, and dense versions on stages where scalar elimination dominates.

The success criterion is an actual reduction in relevant work or memory, with the original differential information preserved. The nilpotence bound alone does not guarantee a practical improvement.

12.6 Compress differential operators, not only ranks

For repeated braid blocks or repeated tangle patterns, can the minimal differential be represented by structured matrices, tensor networks, or recurrences whose composition and final rank are efficiently computable? A correct representation must survive gluing and cancellation without expanding an exponential basis.

The first decisive theorem would establish closure of a useful representation class under those operations with controlled encoded size. Reporting exponentially large ranks in binary is only the starting point.

12.7 Resolve the multi-boundary matching model explicitly

Specify the boundary topology of arbitrary scanner prefixes and prove the appropriate state-space bound for that topology. Where a Catalan estimate is desired, give a disk-frontier certificate or a transformation whose additional width is accounted for.

This question affects both bracket and categorical cost statements. Using the conservative matching count is safe, but understanding which smaller algebra is actually available could give a meaningful parameter improvement.

12.8 Prove a useful multiplicity-profile theorem

Identify a family with $b + \log(\mu + 1) = O(\log^2 n)$ along an efficiently constructible order. The theorem should not follow merely from a small number of strands: the explicit homological output obstruction shows that this is insufficient.

A more realistic class might combine a decomposition certificate, controlled local blocks, and a rule preserving compressed differential structure. Any decomposition must carry a topological gluing proof.

12.9 Implement one complete geometric hierarchy operation

Select a single currently missing transformation, specify its encoded input and output, prove topology preservation, and bound its bit cost and output size. Compressed cutting with provenance, or a precise weak-reduction routine, would be a substantial milestone.

Completing one such primitive is preferable to adding another high-level hierarchy driver whose expensive subroutines remain unspecified. The subsequent theorem should compose its bound with the potential and depth analysis of the whole hierarchy.

12.10 Strengthen certificates and formalize the small trusted core

The free-product reduction, central-lift lemma, accepted conjugacy types, matrix products, and endpoint certificate replay form a compact verification target. A formalization could establish the algebraic parts independently of the much larger general scanner.

The topological input should remain a clearly named theorem: formalizing the stack algorithm does not by itself formalize the closed-three-braid classification. Likewise, a proof checker should distinguish a word certificate, a diagram-equivalence certificate, and a whole-closure Markov certificate.

13 Reproduction and integration

The archive contains the updated Python source, tests, a code patch against the pinned baseline, this article and its source, raw results, experiment scripts, provenance, and checksums. No third-party lecture slides or source papers are redistributed.

Use the included instructions for the exact commands and backend options. The principal checks are:

```
cd Topology/UnknotRecognition/fast
python3 -m unittest discover -s tests -v
python3 -m fastunknot recognize examples/torus_3_5.json
python3 -m fastunknot recognize examples/torus_3_5.json --no-braid
```

The paired benchmark script accepts the original baseline separately so that paired comparisons use unchanged baseline code. The baseline module files can be checked against the recorded Git blob hashes.

The supplied build script runs three pdfLaTeX passes to resolve the table of contents and cross-references; the archive also contains the compiled PDF. The release contains no claimed Lean formalization. Its proof status is conventional mathematical proof, supported by independent implementation checks and exact arithmetic.

14 Conclusions

The continuation supplies an exact complete decision backend whose cost is linear in explicit three-braid word length at the symbol level, and a certified polynomial extension through elementary strand descent. It removes the need to construct Khovanov complexes on those inputs and adds a linear writhe obstruction for general supplied braids.

For the general scanner, the scalar residue identifies multiplicities that every exhaustive pivot strategy must retain at a fixed prefix. The finite radical formula specifies a possible batch algorithm, while the width–multiplicity bound names the parameter still needing control. The geometric route likewise has explicit representation and cost obligations.

These results improve the implementation and narrow the next research targets. They do not establish a general quasi-polynomial complexity bound for the delivered recognizer.

References

- [1] V. Reshetnikov and ProveIt contributors. *Topology/UnknotRecognition: implementations, proposals, and synthesis*. Pinned commit 4e6fe879e5238ec0b134b6d33fdca0b8c7c1c711, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/tree/4e6fe879e5238ec0b134b6d33fdca0b8c7c1c711/Topology/UnknotRecognition>.
- [2] M. Lackenby. *Unknot recognition in quasi-polynomial time*. February 2021 slides; related Oxford lecture announcement, 2021. <https://www.maths.ox.ac.uk/node/38304>;
<https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf>.
- [3] M. Lackenby. *Incompressible surfaces, hierarchies and unknot recognition*. arXiv:2607.23350v1, 2026.
<https://arxiv.org/abs/2607.23350>.
- [4] T. Kelomäki and D. Schütz. *On computational complexity of Khovanov homology*. arXiv:2601.02119v1, 2026. <https://arxiv.org/abs/2601.02119>.
- [5] J. S. Birman and W. W. Menasco. A note on closed 3-braids. *Communications in Contemporary Mathematics* 10, suppl. 1 (2008), 1033–1047. <https://doi.org/10.1142/S0219199708003150>;
<https://arxiv.org/abs/0802.1072>.
- [6] J. S. Birman and H. M. Hilden. Heegaard splittings of branched coverings of S^3 . *Transactions of the American Mathematical Society* 213 (1975), 315–352.
https://www.math.columbia.edu/~jb/hs_of_bcovs.pdf.
- [7] A. Datta. *The braid group B_3 in the framework of continued fractions*. arXiv:2008.02262, 2020.
<https://arxiv.org/abs/2008.02262>.
- [8] A. Datta. *A novel connection between integral binary quadratic forms and knot polynomials*. arXiv:2204.13660, 2022. <https://arxiv.org/abs/2204.13660>.
- [9] D. Bennequin. Entrelacements et équations de Pfaff. *Astérisque* 107–108 (1983), 87–161.
https://www.numdam.org/item/AST_1983__107-108__87_0/.
- [10] H. R. Morton. An irreducible 4-string braid with unknotted closure. *Mathematical Proceedings of the Cambridge Philosophical Society* 93 (1983), 259–261. <https://doi.org/10.1017/S0305004100060540>.
- [11] J. S. Birman and T. E. Brendle. Braids: a survey. In *Handbook of Knot Theory*, Elsevier, 2005, 19–103.
<https://doi.org/10.1016/B978-044451452-3/50003-4>;
<https://www.math.columbia.edu/~jb/Handbook-21.pdf>.
- [12] M. Khovanov. A functor-valued invariant of tangles. *Algebraic & Geometric Topology* 2 (2002), 665–741.
<https://arxiv.org/abs/math/0103190>; <https://msp.org/agt/2002/2-2/agt-v2-n2-p03-p.pdf>.
- [13] D. Bar-Natan. Fast Khovanov homology computations. *Journal of Knot Theory and Its Ramifications* 16 (2007), 243–255. <https://doi.org/10.1142/S0218216507005294>;
<https://arxiv.org/abs/math/0606318>.
- [14] P. B. Kronheimer and T. S. Mrowka. Khovanov homology is an unknot-detector. *Publications Mathématiques de l’IHÉS* 113 (2011), 97–208. <https://doi.org/10.1007/s10240-010-0030-y>;
<https://arxiv.org/abs/1005.4346>.
- [15] M. Crainic. *On the perturbation lemma, and deformations*. arXiv:math/0403266, 2004.
<https://arxiv.org/abs/math/0403266>.